Aussie EcoGuard: Friends & Foes

A Multi-Cloud Serverless Invasive Pests & Pets Monitoring Platform

FIT3184 2026 S2 Assignment 2 (Weight: 40%)

Due date: Monday of Week 14 (23:55, 2 November, 2026)

1. Background

Australia boasts unique biodiversity, but invasive pests such as feral cats, foxes, rabbits, feral pigs, and cane toads severely threaten its native ecosystems. At the same time, domestic pets like stray cats or lost dogs often wander into bushland or residential areas. A machine learning-based, vision-driven platform plays a critical role in biosecurity and community safety, allowing researchers, rangers, and citizens to track invasive species and help locate lost pets. Capturing photos and video behavioural clips of animal sightings provides crucial data for wildlife management and ecological protection.

Imagine trying to find a specific photo of a rare species from last year if it's buried on someone's random hard drive or losing a whole batch of important video recordings because a memory card got corrupted. Data scattered everywhere is data that's hard to use, easy to lose, and impossible to share properly. A good storage system keeps everything safe, backed up, and organised in one central location, so anyone who needs data can find it quickly. Collaboration becomes easier because everyone works from the same organised data, reducing the risk of lost information.

This assignment aims to build a multi-cloud online storage system that lets users store, organise, and retrieve media files using auto-generated and manual tags. The focus of this project is to design a serverless application across at least two major cloud providers (e.g., AWS, GCP/Azure/Oracle) that enables clients to upload media files (images and videos) to cloud storage. Upon upload, the application automatically tags the file with detected species (e.g., domestic pet or invasive pest species like cane toad, fox, and feral pig) using pre-trained ML models, automatically sorts/moves the file, and updates a database for search and retrieval. In addition, it provides a user-friendly, chatbot-powered tool that lets users find their lost domestic pets through the platform.

2. Group Assignment

Most software developers will work in a team at some point in their careers. Therefore, this assignment is a group project designed to prepare students to experience software development as a team. Students will team up in groups of up to 4 to implement the system described above. In this assignment, students need to organise their teams and collective involvement. There is no designated team leader, but teams may establish processes to agree on how to distribute work.

Considering the assignment workload, each group ideally should have 3-4 members; grouping across different labs is allowed. If you want to form a group of 2, please be aware that you need a strong reason/experience/skillset. Please email Jay with supporting evidence for consideration. **Group work is mandatory for this assignment and part of the assessment. Individual submissions are not allowed and won't be assessed (i.e., 0 marks).**

Understanding the dependencies between individual efforts and their successful integration is crucial to the success of the work and for software engineering projects in general. If teams encounter any "issues", they should try to resolve the issues first. In case of an unresolvable issue, please inform the teaching team as soon as possible.

The due date for this assignment is firm, and there is no 'special consideration' for granting an extension (i.e., individual special consideration or extension doesn't apply to a group). **If a team member fails to complete their work for any reason, other team members should take over their tasks.** We will evaluate individual involvement in the project. If a team member fails to satisfy their tasks for non-compelling reasons, they may be penalised heavily, regardless of the project's success.

Assignment Description

Teams should develop a multi-cloud solution that leverages serverless compute, RESTful APIs, AWS Lex chatbot integration, storage, and database services across multiple providers (e.g., AWS, GCP, Oracle, or Azure) to build a system for automated pest/pet identification, lost pet retrieval, and conversational query processing. The application enables users to upload media to cloud storage, automatically trigger serverless classification functions, index results in a highly available database, and interact with the platform via natural language queries or a conversational chatbot interface.

There are no restrictions on which cloud vendors and services you should use. However, you should use **at least two clouds(multi-cloud)**, evaluate the services, and consider the pros and cons of each service your team chooses from different vendors. During the interview, you will be asked to justify your architecture design and decision.
Cost Management: Students are solely responsible for any infrastructure costs incurred during this assignment, until all assessments are completed in Week 15. To eliminate the

expenses, we strongly advise you to proactively utilise free trial offerings and student academy programs offered by the unit and cloud vendors. Furthermore, since this is a collaborative group assignment, your team has the full flexibility to decide how to fairly manage and share any free trial credit among your members.

3. Authentication and Authorisation

Security is one of the most crucial aspects of developing cloud applications. When your application is publicly exposed, you must safeguard your endpoints and resources against unauthorised access and malicious requests. AWS, through its Cognito service, provides a straightforward, secure, and centralised approach to protect your web application.

For user authentication and authorisation, you must use AWS Cognito. Other services can be hosted in other cloud services. You may need to consider how to securely pass AWS Cognito-generated authentication tokens to authorise requests to your secondary cloud provider's API endpoints or serverless functions.

Your application should support the following workflow and features:

- Detect whether a user is authenticated. If the user has not signed in, access to all pages and endpoints except the sign-up service should be blocked, and the user should be redirected to the sign-up page to register a new account.
- For each new account, you need to record the user's email address, first name, last name, and password. Cognito will send an email to new users, asking them to verify their email address and change their temporary password.
- Your application should include a login page that allows users to sign in. After successful authentication, users should be able to upload files, submit queries, view query results, and sign out of the application. All of these services must be protected against unauthorised access.
- You can implement login and sign-up web pages using either the Hosted UI feature in Cognito or your own custom implementation that calls Cognito APIs.
- Uploading files, invoking serverless functions, and accessing the database require fine-grained access control permissions that you need to configure via IAM roles (or equivalent policies in your secondary cloud). It is important to note that IAM roles in AWS Academy have several limitations. Therefore, you should carefully consider how to perform authentication and authorisation in your system while considering these limitations. You can also create a free AWS account with a personal email to bypass this restriction (https://aws.amazon.com/free/).

4. Core Functionalities

4.1 Model Handling

Machine learning and deep learning models are often updated to improve their quality. If a

new version of the pretrained model becomes available, your solution design should be flexible enough to use it without changing your source code/lambda function.

In terms of pre-trained models, you're permitted to use the following options:

1. Utilising MegaDetector from Microsoft OpenSource: https://microsoft.github.io/MegaDetector/, convert the model inferencing to a serverless function
2. Using Ultralytics YOLO (https://docs.ultralytics.com/) in your serverless function
3. Managed services like AWS Rekognition.

As this is not a machine learning unit, you're not required to fine-tune or train a model; you just need to know how to use and deploy them.

To improve your application quality, you can use a chain of models. For example, if model 1 has poor performance in a particular image, you can forward the request to another model. Feel free to use any model you can find on Hugging Face (https://huggingface.co/) or other websites; however, the first/entry model must be one of the options above.

4.2 File Handling

Your solution should provide a mechanism to upload a file to cloud storage (e.g., an S3 bucket or GCS). Uploading a file can be done either through a RESTful API endpoint or directly using cloud SDKs. Whenever a file is uploaded, your system must trigger an event and invoke a serverless function.

- **Deduplication:** Users may upload the same file repeatedly, wasting storage space. Your application should have a mechanism to prevent duplicate uploads using a file checksum.

- **Building Image Thumbnails:** Upon uploading an image, a serverless function should generate a thumbnail for it. Creating a thumbnail involves resizing the original image to a smaller dimension while maintaining its aspect ratio, then compressing it to reduce file size. You can use the OpenCV library for this purpose. You are expected to configure the triggers and grant the required resource permissions (execution roles) for the function.

- **Tagging Files:** Another serverless function should be triggered to read the uploaded file, detect species using the ML model, and save the file type and a list of detected tags, along with the storage URLs for the file and its thumbnail (for images), in a database for future queries.
- **Video files** can be handled by extracting frames as images, then using the image detection model. For consistency and to reduce data processing, please extract one image per 10 seconds (do not try to extract all frames).

4.3 Queries/Functionalities

Your application should provide APIs that allow the following queries:

- **Find images and videos based on pest/pet tags:** The user can send a JSON-based query to request URLs of files containing specific tags with minimum counts (e.g., {"fox": 2}, {"feral pig": 1, "cane toad": 3}, which shows all files with >=2 foxes in the first case and >=1 feral pig AND >=3 cane toads in the second). RESTful endpoints will return thumbnails and video URLs using logical AND operations on tags.

- **Find files based on species or lost pet search feature:** Users can query for specific pest or pet species (e.g., {"cat"}). Furthermore, the system must support a "Search for lost pet" feature where users can input an image of a missing pet (or specify tags like breed/color) to search recent public sightings across the database.

- **Find files based on the thumbnail's URL:** Users can input the URL of a thumbnail into the system and receive the corresponding full-size image URL.

- **Find files based on the tags of a file:** The user can send a file as part of an API call. The system discovers the list of all species (tags) in the uploaded file, then finds all files in the database that contain that set of tags. In response, return the list of URLs for the matching media files. To protect users' privacy, the file uploaded for query purposes should **not** be added to the database or permanently stored.

- **Manual addition or removal of tags with bulk tagging:** Your solution should provide a POST API that enables end-users to manually add or remove tags from files. The JSON message sent must include a list of URLs, the tags to modify, and an "operation" key set to 1 (for adding a tag) or 0 (for removing a tag). If a tag is not included in the list of tags requested for deletion, simply ignore it.

- **Delete files:** The user can send a list of URLs to an API, and the system should remove the files and their thumbnails from storage and all relevant entries from the database.

- **Chatbot Integration for Natural Language Queries:** The application must integrate a chatbot interface that accepts natural language queries (e.g., "Show me all photos of foxes spotted yesterday" or "Help me find a lost ginger cat"). The chatbot should parse intent/entities, trigger backend serverless query functions, and present matching media links to the user.

4.4 Tag-based Notifications

Users can receive email notifications or updates related to files based on specific tags. For example, users can receive notifications when new images with specific species are added or updated to the system. You can use AWS SNS or a similar serverless messaging equivalent on your secondary cloud for this purpose.

5. User Interface

A user interface (UI) makes your system easier and more enjoyable to use. You can design a simple web-based UI. The UI must support all core functionalities, including uploading files, submitting queries, viewing query results, adding/removing tags, etc.

You have the option to either not create a UI for the application or to implement a partially implemented UI for some parts. However, if you opt for this, you need to consider how to demonstrate the core functionalities during the demo.

6. Demo and Final Reports

You should write an individual and team report on the developed application. Use a highly legible font such as Arial, Helvetica, or Times New Roman with a font size of 12 points.

6.1 Team Report (Maximum 1000 words)

Please prepare a team report based on the following guidelines:

- Include a multi-cloud architecture diagram in the team report. For the architecture diagram, you must use official architecture icons for all involved cloud providers. You can use draw.io, mermaid.js.org or any other tools for this purpose.

- Include a table to describe the role of each team member in your team report. You shall provide a three-column table that shows the student's name and ID, the percentage of contribution, and the elements of the project the member contributed (a maximum of 100 words per member).
- The report includes a simple user guide for testing your application.
- Your report should include a link to the source code (GitHub, Gitlab, or Bitbucket). **All students must commit their code to the code repository rather than delegate this to a single team member. This can provide an evidence base if teams run into "issues" and show how you contributed.** You should share your repository with the whole teaching team (emails can be found on Moodle → Additional Information). Please do not use a public repository to avoid any plagiarism.
- Note that the tables, architecture diagram, screenshots of UI, and references are excluded from the word count limit.

6.3 Individual Report - 500 words

Please prepare an individual report based on the following guidelines:

- In your individual report, you should describe your role in the project and how you contributed to the delivery of the multi-cloud system (150 words).
- Include a personal reflection on the teamwork experience of working with other individual team members in your group. Comment on your teammates' work, challenges the entire team faced, and how well the team worked (150 words).
- Note that the individual report should be done with little or no consultation with other team members. Please be careful about plagiarism and collusion.

7. Submission

You need to submit the following via Moodle:

- Your **team report** in PDF format. Only **one team member** submits this report. Please do not forget to include your team members' names and student IDs. One submission per team!

- Your individual report in PDF format. Every team member should submit this report. Please do not forget to include your name and student ID.

8. Demonstration Schedule, Venue and Peer Assessment

- Demonstration is a mandatory component, and the marking process only happens during the demonstration (**No demo = No marks!**)
- Teams are required to demonstrate the working application via Zoom.
- All team members must be present at the demonstration time slot to answer questions.
- The presentation should start with the architecture of your application as described in your report, along with the design and implementation choices you made.
- Each team has up to 3 minutes to present its architecture.
- This is followed by a demo of the application, as per the examiners' guidelines, for up to 15 minutes, and Q&A. We will announce the demo date and time closer to the due date.
- We will ask you to select a time slot during the allocated hours to join a Zoom meeting.
- **As mentioned earlier, this is a group assignment. If one of the team members is experiencing difficulties or has special circumstances, the rest of the team should proceed with the project without that member.** This aligns with industry practices, where the entire project cannot be delayed because of a team member's departure or

absence.

- For late submission, the faculty's standard policy applies. That is up to **7 days** with a 5% penalty on each day, rounded up. After 7 days, your submission won't be assessed (i.e., no mark).
- At the end of the session, teams will conduct peer assessments of their fellow members' contributions and involvement in the project. **Those who do not actively contribute to the group project may be penalised by up to 50% off their group mark.**

  This feedback may be gathered through a Google Form. Each team member is required to submit a **confidential report** detailing their role in the project and their experiences collaborating with team members.

  These reports will be treated with the utmost confidentiality, along with your final report. The purpose of this evaluation is not to assign blame, but rather to ensure that all team members have an opportunity to provide comments and to guarantee active participation from everyone.

- We encourage all team members to collaborate effectively to achieve the project's objectives and optimise your team's performance. Should you observe a team member encountering difficulties with their tasks, please offer assistance. If a team member fails to fulfil their assigned tasks or deviates from the project's expectations, we urge you to promptly inform the teaching team. Alternatively, you can utilise this opportunity to bring the matter to our attention.

9. Generative AI Statement

As per the University's policy on the guidelines and practices pertaining to the usage of Generative AI:
AI & Generative AI tools may be used unrestricted within this assessment. Where used, AI must be used responsibly, clearly documented and appropriately acknowledged (see Learn HQ).
Any work submitted for a mark must:

- Represent a sincere demonstration of your human efforts, skills and subject knowledge that you will be accountable for (students should be able to understand and modify their code if generated using GenAI tools).
- Adhere to the guidelines for AI use set for the assessment task.
- Reflect the University's commitment to academic integrity and ethical behaviour.

Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.

The teaching team encourages students to apply their own critical thinking and reasoning skills when working on the assessments with an assistant from GenAI. Generative AI tools may produce inaccurate content, which could negatively impact students' comprehension of big data topics.

10. Where to get help

If you need any help or clarification, please ask in the Ed forum. Please bear in mind staff members have boundaries on what they're allowed to answer for assignment-related questions. Instead, please consider framing your questions as general technical questions. In addition to the Ed forum, we have at least 2 consultation sessions each week.